

Proyecto final

ProsperApp

Herramienta para la gestión de proyectos de software personales — Informe técnico

Asignatura	Bases de Datos
Integrantes	Camilo Portilla (2418800) Jhon Steven Angulo (2415995) Javier Alexander Ramirez (2325151)
Profesor	Jefferson Amado Peña Torres
Semestre	2026-2
Lugar	Santiago de Cali, Colombia

Universidad del Valle — Escuela de Ingeniería de Sistemas y Computación

Contenido

1. Introducción y objetivo del proyecto
2. Descripción de la plataforma
3. Modelo Entidad-Relación
4. Modelo Relacional
5. Normalización
6. Reglas de negocio implementadas con triggers
7. Índices y rendimiento
8. Scripts y datos simulados
9. Arquitectura de la aplicación
10. Estado del proyecto y proceso de desarrollo
11. Conclusiones
12. Enlaces del proyecto

1. Introducción y objetivo del proyecto

Este documento presenta el diseño e implementación de ProsperApp, desarrollado como proyecto final del curso de Bases de Datos de la Universidad del Valle. El objetivo del proyecto es aplicar los conceptos y la metodología vistos en el curso, diseñando e implementando una aplicación cuyo núcleo es una base de datos relacional, integrando funcionalidades que presentan información relacionada con los datos almacenados durante la interacción con usuarios reales o ficticios.

De manera específica, el desarrollo buscó: entender un problema real de manejo de información, levantar los requerimientos funcionales y no funcionales de la aplicación, planificar la implementación en un grupo de desarrollo colaborativo, diseñar una base de datos relacional segura y eficiente, implementar un prototipo funcional, y documentar el proceso mediante este informe.

2. Descripción de la plataforma

ProsperApp nace de una necesidad parcialmente satisfecha: un desarrollador que tiene una idea y quiere implementarla —ya sea una prueba de concepto, un experimento con nueva tecnología, o simplemente automatizar una tarea repetitiva— no siempre necesita un equipo grande, pero sí necesita herramientas para monitorear el progreso, gestionar recursos y mantener el desarrollo dentro de los plazos establecidos.

ProsperApp permite la gestión de proyectos de software personales: proyectos ideados, diseñados e implementados por una sola persona, con pocos colaboradores y/o pocos usuarios. Desde la vista principal, el usuario encuentra un tablero tipo Kanban dividido en secciones configurables (mínimo 1, máximo 6 — por ejemplo Backlog, Doing, Completed, Release). Dentro de cada sección se organizan las funcionalidades: tarjetas que representan una historia de usuario, y que además pueden documentar una descripción detallada, notas de diseño, fragmentos de código, un checklist de subtarefas y las decisiones técnicas tomadas durante su desarrollo, junto con su justificación.

3. Modelo Entidad-Relación

El modelo ER se construyó en notación Crow's Foot: las entidades se representan como tablas con sus atributos, las llaves primarias y foráneas se etiquetan explícitamente, y la cardinalidad de cada relación se indica sobre cada extremo de la conexión (uno o muchos).

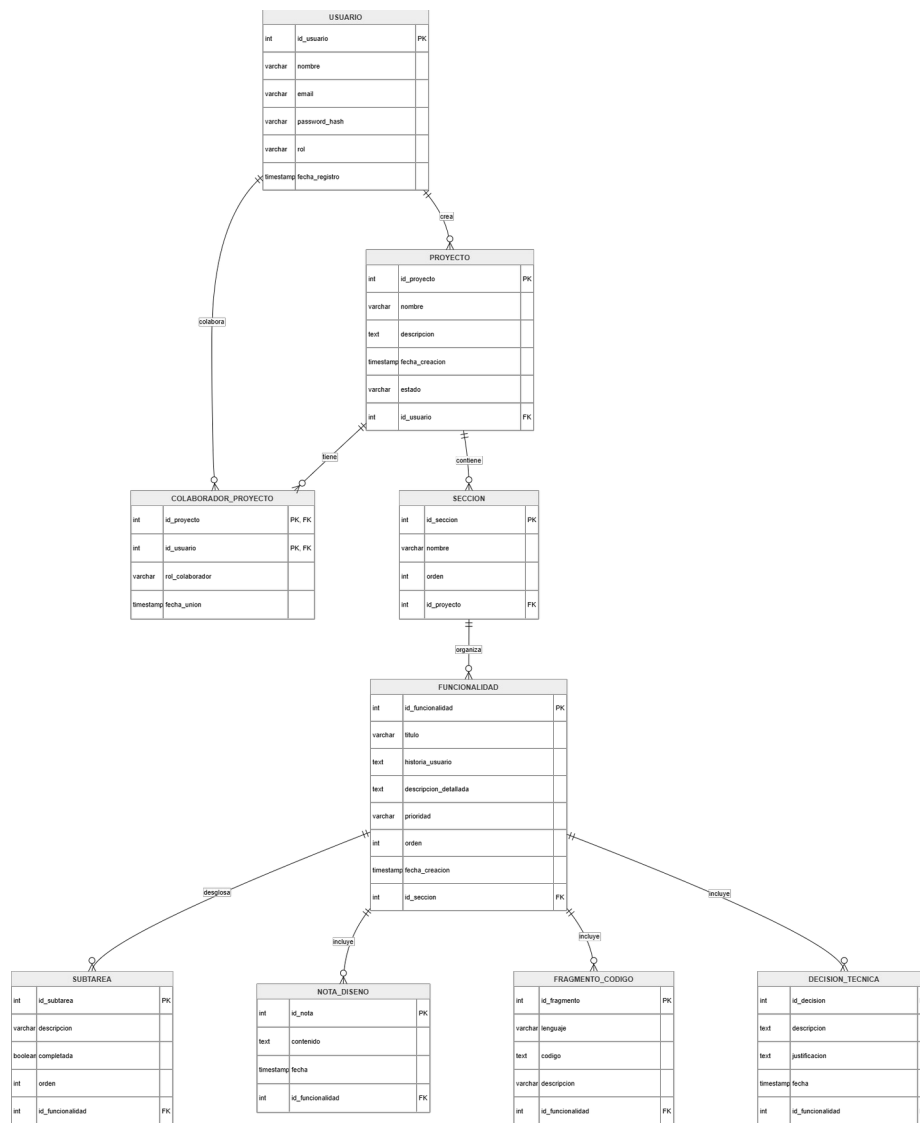


Figura 1. Modelo Entidad-Relación de ProsperApp (9 entidades)

3.1 Entidades principales

Entidad	Descripción
usuario	Persona registrada en la plataforma; puede ser dueña de proyectos y/o colaborar en proyectos ajenos.
proyecto	Proyecto de software personal, con un único dueño (id_usuario).
seccion	Columna del tablero Kanban de un proyecto (entre 1 y 6 por proyecto).
funcionalidad	Tarjeta/historia de usuario dentro de una sección.
subtarea	Ítem del checklist de una funcionalidad.
nota_diseño	Nota de diseño asociada a una funcionalidad (puede haber varias).
fragmento_codigo	Fragmento de código asociado a una funcionalidad (puede haber varios, en distintos lenguajes).
decision_tecnica	Decisión técnica tomada sobre una funcionalidad, junto con su justificación.

3.2 Relaciones y cardinalidades

Relación	Entidades	Cardinalidad	Descripción
crea	usuario — proyecto	(0,N) — (1,1)	Un usuario crea cero o varios proyectos; cada proyecto pertenece exactamente a un usuario (el dueño).
colabora	usuario — proyecto	(0,N) — (0,N)	Relación muchos a muchos, materializada en la tabla colaborador_proyecto, con los atributos propios rol_colaborador y fecha_union.
tiene	proyecto — sección	(1,1) — (1,6)	Cada sección pertenece a un único proyecto; cada proyecto tiene entre 1 y 6 secciones (regla de negocio validada con triggers).
contiene	sección — funcionalidad	(1,1) — (0,N)	Cada funcionalidad vive en una única sección a la vez; una sección puede tener cero o varias funcionalidades.
tiene / documenta / incluye / registra	funcionalidad — {subtarea, nota_diseno, fragmento_codigo, decision_tecnica}	(1,1) — (0,N)	Cada funcionalidad puede tener cero o varios elementos de cada tipo; cada elemento pertenece a una única funcionalidad.

4. Modelo Relacional

El modelo ER se mapeó directamente al modelo relacional: cada entidad se convirtió en una tabla, y la relación muchos a muchos colabora se materializó como la tabla puente colaborador_proyecto, con clave primaria compuesta y sus propios atributos. A continuación se documentan las 9 tablas resultantes.

usuario

Columna	Tipo	Restricciones
id_usuario	SERIAL	PRIMARY KEY
nombre	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
rol	VARCHAR(20)	NOT NULL, DEFAULT 'usuario', CHECK IN ('admin','usuario')
fecha_registro	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

proyecto

Columna	Tipo	Restricciones
id_proyecto	SERIAL	PRIMARY KEY
nombre	VARCHAR(150)	NOT NULL
descripcion	TEXT	
fecha_creacion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Columna	Tipo	Restricciones
estado	VARCHAR(20)	NOT NULL, DEFAULT 'activo', CHECK IN ('activo','completado','archivado')
id_usuario	INT	NOT NULL, FK → usuario(id_usuario) ON DELETE CASCADE

colaborador_proyecto (tabla puente N:M)

Columna	Tipo	Restricciones
id_proyecto	INT	PK (compuesta), FK → proyecto(id_proyecto) ON DELETE CASCADE
id_usuario	INT	PK (compuesta), FK → usuario(id_usuario) ON DELETE CASCADE
rol_colaborador	VARCHAR(20)	NOT NULL, DEFAULT 'editor', CHECK IN ('editor','lector')
fecha_union	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

seccion

Columna	Tipo	Restricciones
id_seccion	SERIAL	PRIMARY KEY
nombre	VARCHAR(50)	NOT NULL
orden	INT	NOT NULL, CHECK (orden > 0)
id_proyecto	INT	NOT NULL, FK → proyecto(id_proyecto) ON DELETE CASCADE

Restricción adicional: UNIQUE (id_proyecto, orden) — evita dos secciones con el mismo número de orden dentro del mismo proyecto.

funcionalidad

Columna	Tipo	Restricciones
id_funcionalidad	SERIAL	PRIMARY KEY
titulo	VARCHAR(150)	NOT NULL
historia_usuario	TEXT	Formato "Como... quiero... para..."
descripcion_detallada	TEXT	
prioridad	VARCHAR(20)	DEFAULT 'media', CHECK IN ('alta','media','baja')
orden	INT	NOT NULL, DEFAULT 0, CHECK (orden >= 0)
fecha_creacion	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
id_seccion	INT	NOT NULL, FK → seccion(id_seccion) ON DELETE CASCADE

subtarea

Columna	Tipo	Restricciones
id_subtarea	SERIAL	PRIMARY KEY
descripcion	VARCHAR(255)	NOT NULL
completada	BOOLEAN	NOT NULL, DEFAULT FALSE
orden	INT	NOT NULL, DEFAULT 0, CHECK (orden >= 0)
id_funcionalidad	INT	NOT NULL, FK → funcionalidad(id_funcionalidad) ON DELETE CASCADE

nota_diseno

Columna	Tipo	Restricciones
id_nota	SERIAL	PRIMARY KEY
contenido	TEXT	NOT NULL
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
id_funcionalidad	INT	NOT NULL, FK → funcionalidad(id_funcionalidad) ON DELETE CASCADE

fragmento_codigo

Columna	Tipo	Restricciones
id_fragmento	SERIAL	PRIMARY KEY
lenguaje	VARCHAR(50)	
codigo	TEXT	NOT NULL
descripcion	VARCHAR(255)	
id_funcionalidad	INT	NOT NULL, FK → funcionalidad(id_funcionalidad) ON DELETE CASCADE

decision_tecnica

Columna	Tipo	Restricciones
id_decision	SERIAL	PRIMARY KEY
descripcion	TEXT	NOT NULL
justificacion	TEXT	
fecha	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
id_funcionalidad	INT	NOT NULL, FK → funcionalidad(id_funcionalidad) ON DELETE CASCADE

Todas las llaves foráneas usan ON DELETE CASCADE de forma intencional: al eliminar un proyecto, se eliminan automáticamente sus secciones, funcionalidades y todo el contenido asociado (subtareas, notas, código, decisiones), sin dejar registros huérfanos.

5. Normalización

El modelo relacional se diseñó cumpliendo la Tercera Forma Normal (3FN), evitando redundancia de datos:

Primera Forma Normal (1FN)

Todos los atributos de cada tabla son atómicos. En particular, se evitó modelar "notas", "fragmentos de código" y "decisiones técnicas" como columnas de texto repetidas o concatenadas dentro de funcionalidad; en su lugar, cada una es su propia tabla (nota_diseño, fragmento_codigo, decision_tecnica), lo que permite que una funcionalidad tenga varias notas, fragmentos o decisiones sin violar la atomicidad ni generar grupos repetitivos.

Segunda Forma Normal (2FN)

Todas las tablas con llave primaria simple cumplen 2FN de forma directa. El caso a revisar es colaborador_proyecto, cuya llave primaria es compuesta (id_proyecto, id_usuario): sus atributos no clave (rol_colaborador, fecha_union) dependen de la combinación completa de ambas columnas, no de una sola, por lo que no hay dependencias parciales.

Tercera Forma Normal (3FN)

No existen dependencias transitivas: ningún atributo no clave depende de otro atributo no clave. Por ejemplo, el dueño de un proyecto (id_usuario) se almacena únicamente en proyecto, y no se repite en seccion ni en funcionalidad (se obtiene navegando la cadena de llaves foráneas). Esto evita que, por ejemplo, cambiar el dueño de un proyecto implique actualizar múltiples tablas.

Decisión de diseño clave: separar notas de diseño, fragmentos de código y decisiones técnicas en tablas propias (en vez de columnas únicas en funcionalidad) fue la decisión de normalización más importante del modelo: permite cardinalidad 1:N real para cada tipo de contenido, evita redundancia y mantiene la tabla funcionalidad enfocada en los datos que le son propios.

6. Reglas de negocio implementadas con triggers

Algunas reglas del enunciado no se pueden expresar con una restricción CHECK convencional, porque un CHECK solo evalúa los valores de la fila que se está insertando, y no puede contar cuántas filas relacionadas ya existen en otra tabla. Por esta razón, tres reglas de negocio se implementaron como triggers.

6.1 trg_max_secciones — máximo 6 secciones por proyecto

Antes de insertar una nueva sección, cuenta cuántas secciones tiene ya el proyecto; si son 6 o más, rechaza la inserción.

```
CREATE TRIGGER trg_max_secciones
BEFORE INSERT ON seccion
FOR EACH ROW
EXECUTE FUNCTION validar_max_secciones();
```


6.2 trg_min_secciones — mínimo 1 sección por proyecto

Antes de borrar una sección, valida que al proyecto le quede al menos una. Esta regla tuvo un problema durante las pruebas manuales, documentado aquí porque refleja directamente un "factor de calidad" del diseño:

Bug encontrado y solución: la primera versión del trigger bloqueaba el borrado de un proyecto completo. Al eliminar un proyecto, el ON DELETE CASCADE de la FK proyecto → sección dispara un DELETE por cada sección, una por una. Al llegar a la última sección restante, el trigger la rechazaba con "no puede quedar sin secciones", cancelando el borrado del proyecto completo — la operación que sí se quería realizar.

La solución usa `pg_trigger_depth()`, función de PostgreSQL que indica si el trigger se está ejecutando anidado dentro de otro proceso. Si la profundidad es mayor a 1, el borrado viene de una cascada (por ejemplo, se está eliminando el proyecto completo) y el trigger deja pasar la operación sin validar; si es el primer nivel, es un borrado directo de una sección individual y sí se valida el mínimo.

```
IF pg_trigger_depth() > 1 THEN
  RETURN OLD; -- viene de un borrado en cascada: no bloquear
END IF;
-- si no, sí valida que no sea la última sección del proyecto
```

6.3 trg_no_autoinvitar — el dueño no puede ser su propio colaborador

Antes de insertar una fila en `colaborador_proyecto`, verifica que el usuario que se está agregando no sea el mismo dueño del proyecto (dato que ya tiene acceso total por ser `proyecto.id_usuario`).

```
IF NEW.id_usuario = id_dueno THEN
  RAISE EXCEPTION 'El dueño del proyecto no puede agregarse como colaborador';
END IF;
```

7. Índices y rendimiento

Se crearon índices sobre las columnas que se filtran con mayor frecuencia en las consultas del dashboard y el tablero Kanban (principalmente llaves foráneas), para que esas búsquedas sigan siendo rápidas a medida que crece el volumen de datos:

- `idx_proyecto_usuario` (`proyecto.id_usuario`)
- `idx_seccion_proyecto` (`seccion.id_proyecto`)
- `idx_colaborador_usuario` (`colaborador_proyecto.id_usuario`)
- `idx_funcionalidad_seccion` (`funcionalidad.id_seccion`)
- `idx_subtarea_funcionalidad` (`subtarea.id_funcionalidad`)
- `idx_nota_funcionalidad` (`nota_diseno.id_funcionalidad`)
- `idx_fragmento_funcionalidad` (`fragmento_codigo.id_funcionalidad`)
- `idx_decision_funcionalidad` (`decision_tecnica.id_funcionalidad`)

8. Scripts y datos simulados

El repositorio incluye tres scripts SQL en database/:

- `ddl.sql` — crea las 9 tablas, sus restricciones, los 3 triggers y los 8 índices descritos en este informe. Se puede ejecutar varias veces sin error gracias a los `DROP TABLE IF EXISTS ... CASCADE` al inicio.
- `dml.sql` — datos simulados para pruebas: usuarios, proyectos, secciones, funcionalidades y su contenido asociado.
- `queries.sql` — 13 consultas documentadas que representan los accesos a datos que necesita la aplicación (listado de proyectos de un usuario, tablero completo de un proyecto, detalle de una funcionalidad con sus subtareas/notas/código/decisiones, etc.).

Ambos scripts (`ddl.sql` y `dml.sql`) se ejecutan automáticamente al crear el contenedor de PostgreSQL en Docker, gracias a la configuración del volumen de inicialización en `docker-compose.yml`.

9. Arquitectura de la aplicación

Capa	Tecnología
Base de datos	PostgreSQL, en contenedor Docker (servicios db y pgadmin)
Backend	Node.js + Express + pg, autenticación con JWT (jsonwebtoken) y contraseñas cifradas con bcryptjs
Frontend	SPA de un solo archivo HTML, sin framework — Tailwind CSS (CDN) + JavaScript vanilla + Font Awesome

La aplicación hace un uso directo del modelo relacional: cada vista (dashboard, tablero Kanban, detalle de funcionalidad) corresponde a una o varias de las 13 consultas documentadas en `queries.sql`, que se ejecutan contra las 9 tablas descritas en este informe. Por ejemplo, mover una tarjeta entre columnas del tablero no crea un registro nuevo: actualiza el valor de `id_seccion` en la fila de funcionalidad correspondiente.

Para la protección de la información personal, las contraseñas nunca se almacenan en texto plano (se guardan como `password_hash` usando `bcrypt`), y el acceso a los datos de cada usuario se controla mediante autenticación JWT: cada solicitud al backend debe incluir un token válido, y las consultas filtran los datos según el usuario autenticado y su relación con el proyecto (dueño o colaborador).

El frontend además incluye un modo Demo (DML), que simula el uso de la aplicación con datos en memoria sin necesidad de backend ni base de datos real —útil para pruebas rápidas de interfaz— y un modo BD PostgreSQL Real, que sí opera contra la base de datos a través de la API.

10. Estado del proyecto y proceso de desarrollo

A la fecha de este informe, el proyecto cuenta con:

- DDL completo (9 tablas, triggers e índices) corriendo en Docker
- Datos simulados de prueba y 13 consultas documentadas
- Aplicación funcionando de punta a punta: registro/login, tablero Kanban con drag and drop, checklist de subtareas, notas de diseño, fragmentos de código, decisiones técnicas y gestión de colaboradores
- Eliminación de proyectos (con confirmación), disponible solo para el dueño

- Soporte como aplicación web progresiva (PWA), instalable en dispositivos móviles
- Repositorio en GitHub con todo el código fuente y la documentación

El proceso de desarrollo evidenció la importancia de probar las reglas de negocio en escenarios compuestos (como el borrado en cascada de un proyecto completo), no solo en el caso simple para el que fueron escritas inicialmente — de ahí el ajuste documentado en la sección 6.2.

11. Conclusiones

El desarrollo de ProsperApp permitió aplicar de forma práctica los conceptos centrales del curso de Bases de Datos: modelado entidad-relación, mapeo al modelo relacional, normalización, y el uso de restricciones y triggers para reglas de negocio que van más allá de lo que un CHECK simple puede expresar. La separación de notas, fragmentos de código y decisiones técnicas en tablas independientes fue la decisión de diseño que más impactó tanto la normalización del modelo como la flexibilidad real de la aplicación. El caso del trigger de mínimo de secciones, en particular, mostró la importancia de considerar los efectos en cascada al validar reglas de negocio a nivel de base de datos.

12. Enlaces del proyecto

A continuación se listan los enlaces de referencia del proyecto: el video de demostración de la aplicación funcionando, y el repositorio de código fuente completo.

Video de demostración (Google Drive):

https://drive.google.com/drive/folders/1_zbg3WPszHcsdzGKCSbi0mstODye5sa?usp=drive_link

Repositorio de GitHub: <https://github.com/MrStv0012/Prosperapp-BDProyect/blob/main/docs/Diagrama%20DDL.drawio.png>